

A Consolidated Overview of Frameworks, Libraries, and Tools for Machine Learning and Computer Vision

Tobias Schlosser 

Dresden University of Technology
01307 Dresden, Germany

`{firstname.lastname}@tu-dresden.de`

Michael Friedrich 

Chemnitz University of Technology
09107 Chemnitz, Germany

`{firstname.lastname}@cs.tu-chemnitz.de`

August 24, 2026

Abstract

Progress in machine learning (ML) and computer vision (CV) depends increasingly not only on algorithmic advances but also on the software ecosystems required for rapid experimentation, reproducible evaluation, and reliable deployment. This manuscript presents a consolidated overview of tools used throughout the ML/CV lifecycle, encompassing programming languages and development environments, general-purpose scientific computing libraries, core ML frameworks, and specialized CV toolkits for tasks such as detection and segmentation. The discussion follows practical pipeline stages and examines how these components interact in real projects, from data handling and preprocessing through model development, training, profiling, benchmarking, experiment management, and deployment engineering. Beyond cataloguing tools, the manuscript emphasizes criteria that often determine long-term success, including ecosystem maturity, interoperability, hardware acceleration, scalability, and maintainability. It further relates current trends—notably transformer-centered modeling, greater automation of tuning and training workflows, and privacy- and compliance-aware learning—to established engineering practices such as environment pinning, containerization, version control, testing, and monitoring. The resulting practitioner-oriented reference is intended to help researchers and engineers assemble sustainable end-to-end ML/CV software stacks across application domains.

Table of Contents

1	Introduction and Motivation	4
1.1	Emerging trends	4
1.2	Challenges	5
1.3	Best practices	6
2	Programming Languages and Development Tooling	7
2.1	Integrated development environments and notebooks	8
2.1	Google Colab	10
2.2	Jupyter Notebook / JupyterLab	10
2.3	Kaggle Notebooks	10
2.4	MATLAB	10
2.5	PyCharm	10
2.6	RStudio	10
2.7	Spyder	11
2.8	Visual Studio	11
2.9	Visual Studio Code (VS Code)	11
2.2	AI coding assistants	12
2.10	Amazon Q Developer	13
2.11	ChatGPT	13
2.12	Claude Code	13
2.13	Cursor	13
2.14	Gemini Code Assist	13
2.15	GitHub Copilot	14
2.16	JetBrains AI Assistant	14
2.17	Tabnine	14
3	General Frameworks, Libraries, and Tools	15
3.1	Dask	16
3.2	Matplotlib	16
3.3	NetworkX	16
3.4	NumPy	16
3.5	OpenGL	16
3.6	pandas	16
3.7	Plotly.py	16
3.8	scikit-image	16
3.9	SciPy	16
3.10	seaborn	16
3.11	statsmodels	16
3.12	SymPy	17
4	Machine Learning Frameworks, Libraries, and Tools	18
4.1	CUDA Toolkit	19
4.2	cuDNN	19
4.3	Hugging Face Transformers	19
4.4	JAX	19
4.5	Keras	19

4.6	LightGBM	19
4.7	Optuna	19
4.8	PyTorch	19
4.9	scikit-learn	19
4.10	TensorBoard	19
4.11	TensorFlow	20
4.12	XGBoost	20
5	Computer Vision Frameworks, Libraries, and Tools	21
5.1	Albumentations	22
5.2	Detectron2	22
5.3	Kornia	22
5.4	MMDetection	22
5.5	OpenCV	22
5.6	Single Shot MultiBox Detector (SSD)	22
5.7	You Only Look Once (YOLO)	22
6	Supporting Tools and Ecosystems	23
6.1	Annotation and labeling tools	23
6.2	Data preprocessing and management	24
6.3	Hyperparameter optimization	24
6.4	Visualization and model interpretability	24
6.5	Deployment and model serving	25
7	Performance and Benchmarking	25
7.1	Performance metrics	26
7.2	Benchmark datasets	27
7.3	Framework benchmarking	27
8	Conclusion and Outlook	28

1 Introduction and Motivation

Machine learning (ML) and computer vision (CV) have become foundational technologies across science and engineering, with applications ranging from medical imaging and industrial inspection to robotics and large-scale information retrieval. Beyond algorithms and hardware, practical progress increasingly depends on software ecosystems that enable efficient experimentation, reproducible evaluation, and dependable deployment. Consequently, the effectiveness of an ML/CV project is often determined by its surrounding tooling: how data are ingested and transformed, how experiments are organized and tracked, how models are trained and profiled on accelerators, and how inference is packaged, served, and monitored in operational systems.

The tooling landscape is nevertheless dynamic and fragmented. Overlapping frameworks, frequent releases of pretrained models, evolving accelerator stacks, and an expanding range of tools for tracking, annotation, and deployment make it difficult to select components that will remain maintainable over time. This manuscript therefore consolidates widely used tools across the ML/CV lifecycle, emphasizing their typical roles, interoperability constraints, and selection criteria such as ecosystem maturity, scalability, hardware support, and long-term maintainability. Its purpose is to orient newcomers while providing experienced practitioners with a structured basis for assembling robust and sustainable software stacks.

1.1 Emerging trends

Transformer-based architectures have become a dominant design pattern across modalities and increasingly shape the surrounding software ecosystem. The broad availability of pretrained models has shifted many workflows from training from scratch toward adaptation and fine-tuning, thereby increasing the importance of model hubs, standardized checkpoints, dataset interfaces, and utilities for efficient inference. As model sizes and context lengths increase, optimized attention kernels, memory-efficient training methods, mixed-precision execution, and distributed parallelism have become routine requirements rather than optional refinements. Multimodal systems extend these demands by integrating text, vision, audio, and structured signals, motivating unified preprocessing pipelines, interoperable representations, and evaluation tools capable of characterizing cross-modal behavior.

Automation is also expanding across the ML stack. Hyperparameter optimization and automated machine learning (AutoML) frameworks reduce the manual effort required to explore architectures, training schedules, and preprocessing choices. This development aligns with a broader transition toward configuration-driven, pipeline-oriented engineering in which experiments are specified declaratively and executed reproducibly on local workstations, servers, or clusters. Standardized model formats and inference runtimes likewise reduce friction when transferring models between frameworks and deployment targets. Such interoperability supports heterogeneous workflows in which training occurs in one framework while inference and serving rely on specialized runtimes optimized for latency, throughput, memory use, or platform constraints.

Privacy, security, and regulatory requirements increasingly influence the engineering of ML/CV systems. Federated learning seeks to retain sensitive data at their originating sites while enabling collaborative model development, requiring orchestration software that coordinates distributed execution, enforces policy constraints, and produces auditable records. On-device inference and edge deployment create related demands for model compression, quantization, hardware-aware optimization, and secure update mechanisms. Collectively, these trends are driving software ecosystems toward stronger support for controlled data access, provenance tracking, governance workflows, and operational monitoring, reflecting the need for modern ML systems to satisfy reliability, safety, and compliance requirements in addition to predictive performance objectives.

1.2 Challenges

Despite the maturity of many libraries, recurring difficulties arise when a working prototype is transformed into a reliable system. Scaling training is rarely achieved by adding graphics processing units (GPUs) alone; it requires coordinated management of accelerator utilization, distributed execution, and data throughput. Bottlenecks can migrate among computation, memory bandwidth, interconnect communication, and input pipelines, while apparently minor decisions concerning batch size, numerical precision, augmentation, or data sharding may determine whether training is stable and efficient. Memory management is especially consequential for contemporary architectures, because long sequences, high-resolution inputs, and large batches can exceed device capacity and necessitate mixed precision, gradient checkpointing, or distributed parallelism.

Reproducibility remains challenging even for experienced teams. Results may vary because of dependency changes, nondeterministic kernels, accelerator-driver differences, or subtle inconsistencies in preprocessing and random-seed control. Experiment records are also frequently incomplete: metrics and final checkpoints may be retained while exact package versions, data snapshots, augmentation settings, preprocessing code, and hardware or runtime configurations are omitted. Reproducing a reported result can therefore become a forensic reconstruction rather than a routine rerun. Reliable practice requires disciplined dependency pinning, environment capture, dataset versioning, deterministic controls where appropriate, and structured tracking of configurations, artifacts, and execution context.

Deployed ML systems operate in dynamic environments and interact with real users, creating failure modes that are not observable in static offline benchmarks. Dataset bias and distribution shift can erode performance after deployment, while rare conditions, adversarial inputs, or changes in acquisition pipelines may expose previously hidden weaknesses. Interpretability and transparency are often necessary both for debugging and for meeting stakeholder expectations, particularly in high-stakes applications. Governance considerations—including privacy, consent, accountability, and traceability—therefore shape how systems are designed, validated, deployed, and monitored throughout their operational lifetime.

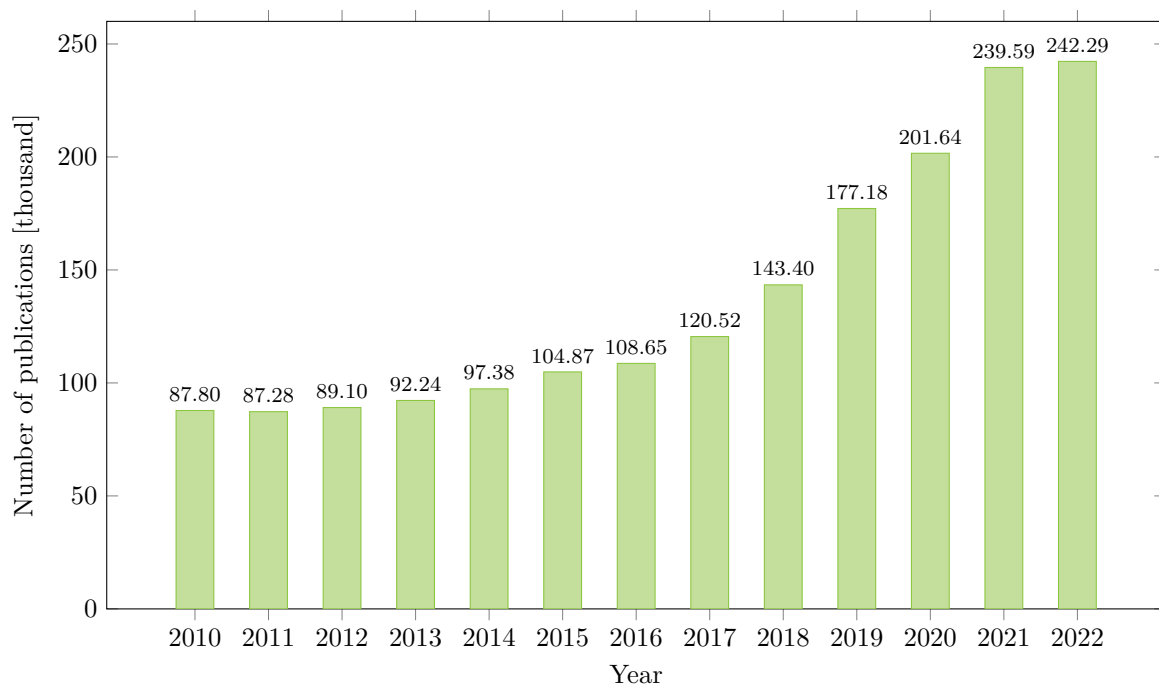


Figure 1: Number of artificial intelligence (AI) publications worldwide from 2010 to 2022 [1, p. 31].

1.3 Best practices

Reliable ML/CV development benefits from treating models as components of a broader software system rather than as isolated research artifacts. This perspective directs attention toward engineering practices that preserve correctness, support collaboration, and make results reproducible. Version control establishes traceability, while code review can identify logical errors in preprocessing, evaluation, and training code before they invalidate experimental conclusions. Continuous integration (CI) complements these practices by automating formatting, static analysis, unit tests, and pipeline-level checks, thereby reducing the risk that incremental changes compromise a growing codebase.

Environment management is equally important because dependency drift is a common cause of irreproducible experiments. Isolated environments and explicit version constraints reduce unintended changes, while containerization with tools such as Docker improves portability across developer workstations, shared servers, and compute clusters. A robust workflow records not only framework and library versions but also the broader execution context, including operating system (OS) details, accelerator drivers, environment definitions, and the configuration files or commands used to launch each experiment.

For deployed models, best practice extends beyond reproducibility to operational reliability. Monitoring is required to detect concept drift, data drift, changes in input quality, and performance regressions that may not be apparent from offline validation. Automated evaluation on curated reference sets, alerts for distributional changes, and structured logging of inputs and outputs can reveal problems early. Testing should additionally address robustness to missing or corrupted inputs, latency and throughput under load, and the correctness of preprocessing, postprocessing, calibration, and thresholding logic.

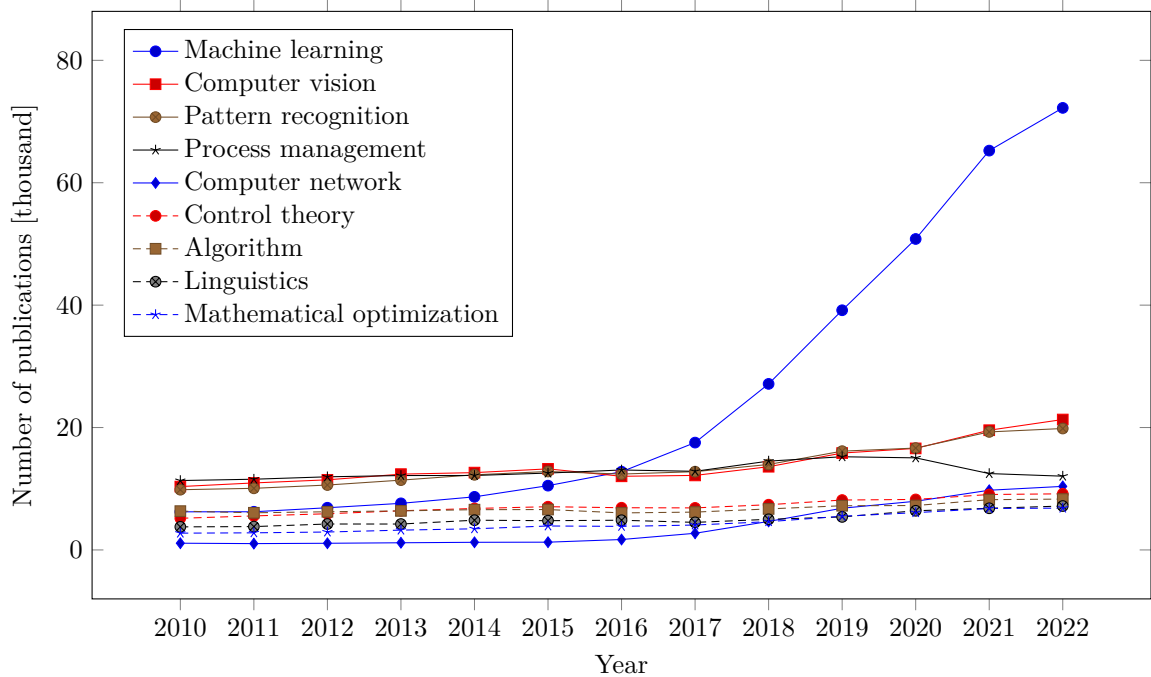


Figure 2: Number of AI publications worldwide from 2010 to 2022 by field of study [1, p. 33].

2 Programming Languages and Development Tooling

Modern ML/CV projects commonly combine several programming languages and execution environments. A typical workflow couples interactive prototyping in a high-level language—most often Python—with performance-oriented backends implemented in compiled languages such as C or C++. Specialized accelerator stacks, including NVIDIA’s Compute Unified Device Architecture (CUDA) and CUDA Deep Neural Network library (cuDNN), provide optimized execution on compatible hardware. As models and datasets grow, engineering concerns become as important as language expressiveness, particularly dependency management, reproducibility, debugging, and portability across workstations, servers, and clusters.

A useful distinction separates the user-facing layer used for experimentation, data preparation, and orchestration from the computational layer that executes kernels efficiently on central processing units (CPUs), GPUs, and tensor processing units (TPUs). Many libraries expose a Python application programming interface (API) while delegating computationally intensive operations to optimized native implementations, thereby combining developer productivity with high performance. When latency, memory footprint, or deterministic execution are strict requirements—as in embedded inference or real-time robotics—deployment often relies more heavily on C/C++, platform-specific runtimes, or compiled model representations.

The surrounding development environment also has a substantial effect on productivity and reproducibility. Dependency isolation and version pinning through virtual or Conda environments reduce “it works on my machine” failures and facilitate later replication. Integrated development environments (IDEs) provide debugging, profiling, navigation, and refactoring capabilities that become increasingly valuable as codebases grow, whereas notebooks remain effective for exploratory analysis, visualization, and executable reporting. Containerization and infrastructure-as-code improve portability from local development to shared infrastructure, while formatting, static analysis, and unit or pipeline testing become indispensable as ML codebases approach the complexity of conventional software systems.

This section therefore examines two environment-level decisions that strongly influence efficient and reproducible work: the interface used to author and execute code, and the integration of artificial intelligence into development workflows. We compare IDEs with notebook-centered environments, emphasizing their respective strengths for ML/CV projects (Table 1), and then review representative AI coding assistants by interaction modality, integration depth, and model support (Table 2). Together, these comparisons support hybrid workflows that use notebooks for exploration and IDEs or complementary tools for modular engineering, testing, optimization, and deployment.

2.1 Integrated development environments and notebooks

Table 1 compares widely used development environments for ML and CV with respect to interaction modality, operating-system support, licensing model, typical use, and AI-assisted functionality. The choice of environment is not merely a matter of individual preference; it affects implementation speed, reproducibility, maintainability, collaboration, and integration with accelerated or remote-computing workflows.

The principal distinction concerns interaction modality. Desktop IDEs such as Visual Studio Code, PyCharm, Spyder, Visual Studio, MATLAB, and RStudio are designed primarily for structured development and sustained project maintenance. By contrast, Jupyter Notebook and JupyterLab, Google Colab, and Kaggle Notebooks emphasize interactive execution and are especially effective for exploratory data analysis (EDA), rapid prototyping, visualization, and experiment documentation. This distinction is consequential in ML/CV projects because early investigation and later engineering frequently require different levels of structure, traceability, and automation [2].

Desktop environments differ further in specialization. Visual Studio Code and PyCharm offer broad support for Python-centered ML pipelines together with mature debugging, project organization, and extension ecosystems. Visual Studio is particularly relevant when CV systems contain performance-critical C++ components, custom CUDA kernels, or native-library integrations. Spyder supports an interactive scientific computing style, MATLAB provides an integrated numerical environment with domain-specific toolboxes, and RStudio is optimized for R-centered statistical and data-science workflows. Tool selection should therefore reflect the project’s programming languages, expected lifetime, computational requirements, and engineering depth.

Notebook environments address a complementary set of requirements. Jupyter Notebook and Jupyter-Lab support transparent, executable analyses and literate programming, but hidden state and non-linear execution can undermine reproducibility unless execution order is carefully controlled. Cloud platforms such as Google Colab and Kaggle Notebooks reduce local setup effort, simplify sharing, and provide managed compute access. Their convenience is balanced by reduced control over runtime configuration, session persistence, resource availability, and quotas. Cloud notebooks are therefore highly effective for prototyping, teaching, and benchmark experiments, but they are rarely sufficient as the sole environment for production-oriented development.

From an infrastructure perspective, most desktop tools support Windows, macOS, and Linux, whereas Visual Studio remains primarily Windows-oriented. Browser-based notebooks are largely OS-independent and therefore lower entry barriers for distributed teams and educational settings. The licensing landscape is similarly diverse: open-source tools coexist with freemium services and commercial environments. This variety enables research groups and industrial teams to balance functionality, support requirements, governance, and cost, but it also makes long-term availability and license compatibility relevant selection criteria.

AI-assisted functionality has become an additional differentiator. Mainstream desktop environments generally provide the most mature integrations for code completion, conversational assistance, refactoring, and repository-aware editing, while notebook support varies across native features, extensions, and external tools. These capabilities can reduce repetitive implementation effort and accelerate navigation through unfamiliar code. They do not, however, replace validation: generated code must still be reviewed for correctness, security, licensing implications, numerical validity, and consistency with the intended experimental protocol.

Overall, Table 1 supports a hybrid strategy. Notebook environments are well suited to data exploration, visual analysis, and early model prototyping, whereas full IDEs become increasingly valuable for modularization, testing, profiling, refactoring, and deployment. Many sustainable ML/CV projects therefore combine both modalities and establish explicit transitions from exploratory notebooks to version-controlled, tested, and reusable software components.

Table 1: Comparison of IDEs and notebook environments for ML and CV by modality, OS compatibility, payment model, typical use, and AI coding assistance.

IDE or notebook	Modality	Typical tasks / notes
Google Colab [3]	Cloud notebook OS: browser-based (OS-independent) Payment model: free tier / paid tiers	Hosted Jupyter-compatible environment with managed CPU, GPU, and TPU resources; effective for rapid experimentation, teaching, sharing, and reproducible demonstrations, but with limited control over runtime persistence and hardware configuration. AI coding assistance: <i>Medium</i> (platform features vary by account, region, and release).
Jupyter Notebook / JupyterLab [4,5]	Notebook environment OS: Windows / macOS / Linux Payment model: free (open source)	Interactive environments for EDA, model prototyping, visualization, and executable documentation; support multiple kernels and languages. JupyterLab provides a more extensible workspace than the document-centered classic Notebook; both require disciplined execution to avoid hidden-state and ordering problems. AI coding assistance: <i>Medium</i> (available through extensions and external assistants).
Kaggle Notebooks [6]	Cloud notebook OS: browser-based (OS-independent) Payment model: free (quota-limited)	Hosted notebook environment integrated with Kaggle datasets, models, and competitions; useful for benchmarking, public demonstrations, and collaborative experimentation, subject to runtime, storage, and accelerator quotas. AI coding assistance: <i>Medium</i> (platform and external assistance may be available).
MATLAB [7]	Numerical-computing environment / IDE OS: Windows / macOS / Linux Payment model: paid licenses / trial and institutional access	Integrated environment for numerical computing, visualization, signal and image processing, simulation, and engineering workflows; extensive domain toolboxes and strong support for prototyping and code generation, but commercial licensing affects accessibility and deployment. AI coding assistance: <i>Medium-High</i> (release- and license-dependent integrated assistance).
PyCharm [8]	Python IDE OS: Windows / macOS / Linux Payment model: free (Community) / paid (Professional)	Full-featured Python IDE with mature debugging, refactoring, testing, environment management, and project navigation; Professional editions add broader scientific and notebook integration, making the environment suitable for larger ML codebases. AI coding assistance: <i>High</i> (vendor and plugin options, depending on subscription).

continued on next page

IDE or notebook	Modality	Typical tasks / notes
RStudio [9]	R-centered scientific IDE OS: Windows / macOS / Linux / browser-based server Payment model: free (open source) / paid professional editions	Integrated environment for R, statistical computing, visualization, package development, Quarto or R Markdown, and reproducible analysis; can interoperate with Python and remote compute environments. AI coding assistance: <i>Medium–High</i> (integrated and extension-based features vary by edition).
Spyder [10]	Scientific Python IDE OS: Windows / macOS / Linux Payment model: free (open source)	MATLAB-like scientific workflow with an IPython console, variable explorer, integrated plots, and debugging; effective for interactive scripting and analysis, but less oriented toward very large multi-module systems. AI coding assistance: <i>Low–Medium</i> (typically external or plugin-based).
Visual Studio [11]	C++ / CUDA IDE OS: Windows Payment model: free (Community) / paid (Professional, Enterprise)	Comprehensive environment for native C++, CUDA, and Windows development; relevant for performance-critical CV components, custom kernels, Python extensions, and advanced debugging or profiling. AI coding assistance: <i>High</i> (integrated and extension-based completion, chat, and refactoring).
Visual Studio Code (VS Code) [12]	Extensible code editor OS: Windows / macOS / Linux Payment model: free	Lightweight editor with extensive support for Python, C/C++, CUDA, notebooks, debugging, Git, Secure Shell (SSH), remote development, and containers; widely used across ML/CV research and engineering. AI coding assistance: <i>High</i> (broad extension ecosystem).

2.2 AI coding assistants

Table 2 compares representative AI coding assistants for ML and CV development by interaction modality, payment model, supported large language models (LLMs), typical tasks, and compatibility with common IDE and notebook environments. In contrast to general environment selection, the principal distinctions are the depth of codebase integration, the scope of editable context, the degree of agentic execution, and the controls available for model selection, data handling, and organizational governance.

Interaction modality provides a useful first distinction. GitHub Copilot, Tabnine, Gemini Code Assist, Amazon Q Developer, and JetBrains AI Assistant operate mainly within supported IDEs and emphasize completion, generation, explanation, and short implementation loops. Cursor provides an AI-centered editor with repository-scale context, while Claude Code emphasizes terminal-based, agentic interaction with files and development tools. ChatGPT occupies a broader conversational role and is often used alongside an IDE or notebook for reasoning, debugging, design, and documentation. These modes are complementary rather than mutually exclusive.

Model access and deployment controls constitute a second differentiator. Some products expose a provider-managed model set, whereas others offer selectable models, enterprise routing, or bring-your-own-key configurations. These distinctions matter when teams must balance task quality, response latency, cost, privacy, auditability, and restrictions on source-code transmission. Because product capabilities and available models evolve rapidly, selection should be based on current documentation and organizational requirements rather than on a fixed feature snapshot.

The task profiles align closely with common stages of ML/CV development. In-editor assistants are effective for repetitive implementation work, including data loaders, augmentation pipelines, training-loop scaffolding, configuration files, utility functions, tests, and documentation. Repository-aware editors and coding agents extend this support to coordinated multi-file changes, dependency updates, debugging, and larger refactorings. Conversational assistants are particularly useful for conceptual and diagnostic work, such as analyzing unstable training, selecting losses or metrics, designing experiments, and explaining unfamiliar implementations.

Compatibility with IDEs and notebooks introduces practical workflow constraints. Visual Studio Code and JetBrains IDEs are the most common integration targets among the listed tools, while Visual Studio remains important for Windows-oriented C++ development. Terminal-based assistants can operate across editors but require careful permission management and review of proposed commands or file changes. Notebook workflows are typically supported either through IDE-hosted notebook editors, platform-specific assistants, or a parallel conversational interface.

Most products use free-tier, subscription, usage-based, or enterprise licensing models, so practical differences arise less from initial availability than from context limits, integration depth, model choice, administrative controls, and data-governance provisions. For many ML/CV teams, a balanced arrangement combines an in-editor assistant for implementation speed with a conversational or agentic tool for broader reasoning and repository-level work, while retaining systematic review, testing, and security controls.

Table 2: Comparison of AI coding assistants for ML and CV development by modality, payment model, model support, typical tasks, and IDE or notebook compatibility.

AI coding assistant	Modality	Typical tasks / notes
Amazon Q Developer [13]	IDE extension / agentic coding assistant Payment model: free tier / paid professional tier	Provides chat, inline suggestions, code transformation, security analysis, and agentic workflows within supported IDEs; particularly relevant to software and infrastructure development on Amazon Web Services. Model support: provider-managed and selectable options, depending on service configuration. Suitable IDEs: VS Code, JetBrains IDEs, Eclipse, and Visual Studio, with feature differences by platform.
ChatGPT [14]	General conversational assistant Payment model: free tier / paid plans	Supports conceptual ML/CV reasoning, debugging, experiment design, code generation, documentation, and analysis of implementation choices; commonly used alongside an IDE or notebook rather than as a single fixed editor. Model support: OpenAI-hosted models, with availability depending on product and plan. Suitable IDEs: usable alongside any IDE or notebook; direct integration depends on the selected product or extension.
Claude Code [15]	Terminal-based coding agent Payment model: subscription and usage-based access	Operates from the command line with repository context and can inspect, modify, test, and explain multi-file codebases; useful for debugging, refactoring, migration, and implementation tasks that span an ML/CV project. Model support: Anthropic-hosted or supported cloud-platform configurations. Suitable IDEs: editor-independent terminal workflow; integrations may be added through supported extensions or development environments.
Cursor [16]	AI-native code editor Payment model: free tier / paid plans	Editor with repository-aware chat, multi-file editing, code generation, and refactoring; useful for converting exploratory research code into maintainable modules and coordinating larger changes across ML/CV repositories. Model support: selectable provider-managed models, depending on plan and configuration. Suitable IDEs: standalone VS Code-derived editor rather than a plugin for PyCharm, Visual Studio, or Spyder.

continued on next page

AI coding assistant	Modality	Typical tasks / notes
Gemini Code Assist [17]	IDE extension / conversational coding assistant Payment model: free tier / paid enterprise tiers	Provides code explanation, generation, completion, and guided workflows within supported editors; applicable to data pipelines, training code, evaluation utilities, and general software engineering. Model support: Google-managed Gemini models, with availability depending on plan and configuration. Suitable IDEs: VS Code and supported JetBrains IDEs, including PyCharm.
GitHub Copilot [18]	IDE extension / in-editor assistant Payment model: free eligibility / paid individual and organizational plans	Provides inline completion, chat, generation, and refactoring assistance; useful for data loaders, training loops, configuration, tests, documentation, and repetitive engineering tasks. Model support: provider-managed or selectable models, depending on product and plan. Suitable IDEs: VS Code, Visual Studio, JetBrains IDEs, and supported notebook workflows.
JetBrains AI Assistant [19]	IDE-integrated assistant Payment model: free and paid capabilities, depending on product configuration	Native assistance for code generation, explanation, refactoring, navigation, tests, and documentation within JetBrains products; particularly relevant to Python-heavy ML development in PyCharm. Model support: JetBrains-managed and configurable provider or local options, depending on the current product. Suitable IDEs: JetBrains IDEs.
Tabnine [20]	IDE extension / in-editor assistant Payment model: free and paid organizational offerings	Provides completion and conversational development support with enterprise-oriented controls; useful for repetitive implementation, API consistency, tests, and documentation where privacy and governance requirements are prominent. Model support: Tabnine-managed and enterprise-configured options, depending on deployment. Suitable IDEs: VS Code, JetBrains IDEs, Visual Studio, and other supported editors.

3 General Frameworks, Libraries, and Tools

Beyond model-training frameworks, ML/CV workflows rely on general-purpose libraries for numerical computing, data management, visualization, graph analysis, image processing, and interactive graphics. These tools support the entire lifecycle, including raw-data conversion and validation, EDA, classical baselines, publication figures, input/output (I/O), and the utility code that connects training, evaluation, and deployment stages.

A typical stack combines array and linear-algebra foundations with tabular and time-series processing, distributed or out-of-core computation, statistical analysis, and visualization. Image and media libraries provide reading, conversion, filtering, augmentation, and batch preprocessing, while graph libraries support relational data and network algorithms. Interactive visualization, symbolic mathematics, and graphics APIs further support communication, derivation, simulation, and synthetic-data generation. The resulting ecosystem is intentionally modular, allowing projects to assemble only the components required by their data and computational constraints.

Although many of these libraries are not ML-specific, they strongly influence experimental velocity and engineering quality. Consistent data representations improve interoperability; deterministic and versioned preprocessing strengthens reproducibility; and mature, well-tested implementations reduce the risk of subtle numerical or data-handling errors. Tool selection at this layer therefore has direct consequences for the reliability of subsequent modeling and benchmarking.

Table 3: General frameworks, libraries, and tools.

Framework, library, or tool	Modality	Typical tasks / notes
Dask [21]	Parallel / distributed computing	Parallel and out-of-core arrays, DataFrames, task graphs, and distributed execution; useful when NumPy or pandas workloads exceed a single core or available memory while retaining familiar Python interfaces.
Matplotlib [22]	Visualization	General-purpose plotting for EDA, debugging, training curves, confusion matrices, and publication-quality figures; highly configurable and widely used as a backend for higher-level libraries.
NetworkX [23]	Graph analysis	Flexible graph data structures, algorithms, generators, and I/O for network analysis and prototyping; prioritizes usability and interoperability over very large-scale graph processing.
NumPy [24]	Numerical computing	Core multidimensional arrays, vectorized operations, linear algebra, random sampling, and numerical utilities; common interchange representation throughout Python-based ML/CV workflows.
OpenGL [25]	Graphics / three-dimensional (3D) rendering	Real-time rendering for simulation, synthetic-data generation, and interactive visualization of images, meshes, point clouds, and volumes; commonly accessed through higher-level wrappers or engines.
pandas [26]	Tabular and time-indexed data	DataFrame-based loading, cleaning, joining, aggregation, missing-value handling, feature engineering, and metadata management; productive for in-memory workloads but potentially limiting at very large scale.
Plotly.py [27]	Interactive visualization	Browser-based interactive charts, dashboards, and exploratory graphics; useful for communicating experiment results and constructing interactive analytical interfaces.
scikit-image [28]	Image processing	NumPy-compatible algorithms for filtering, morphology, segmentation, restoration, measurement, color processing, and geometric transformation; suitable for scientific imaging and classical preprocessing.
SciPy [29]	Scientific computing	Numerical optimization, integration, interpolation, signal processing, sparse matrices, spatial algorithms, and statistics; supports classical methods, calibration, filtering, and numerical validation.
seaborn [30]	Statistical visualization	High-level statistical graphics built on Matplotlib; efficient for distributions, categorical comparisons, relationships, and diagnostic summaries with pandas integration.

continued on next page

Framework, library, or tool	Modality	Typical tasks / notes
statsmodels [31]	Statistical modeling	Interpretable regression, generalized linear models, time-series methods, hypothesis tests, confidence intervals, and diagnostic outputs; emphasizes inference and transparent baselines.
SymPy [32]	Symbolic mathematics	Exact algebra, calculus, equation solving, simplification, and code generation; useful for analytic derivations, independent verification, and gradient checks on tractable examples.

4 Machine Learning Frameworks, Libraries, and Tools

Machine-learning tooling spans classical statistical and tree-based algorithms, differentiable programming systems, and large-scale deep-learning frameworks. Modern frameworks combine automatic differentiation with efficient tensor execution on accelerators and support for distributed training. Surrounding ecosystems add pretrained models, data pipelines, training abstractions, hyperparameter optimization, experiment tracking, compilation, and deployment pathways.

Several criteria repeatedly determine framework suitability. Ecosystem maturity and community support affect stability and integration breadth; hardware acceleration governs throughput and memory efficiency; and distributed-training capabilities determine how readily workloads scale across devices or nodes. Production readiness becomes decisive when models must be serialized, served, monitored, and updated reliably. Developer ergonomics—particularly API consistency, debuggability, and configuration clarity—likewise substantially affect day-to-day productivity and maintainability.

Practical stacks are frequently hybrid. `scikit-learn`, `XGBoost`, and `LightGBM` provide strong and often interpretable baselines for structured data; `PyTorch`, `TensorFlow`, `Keras`, and `JAX` support differentiable model development; Hugging Face Transformers supplies reusable pretrained architectures; and Optuna systematizes hyperparameter search. Common data representations, standardized preprocessing, and reliable export mechanisms are therefore essential for keeping experimentation and deployment coherent across tools.

Table 4: Machine learning frameworks, libraries, and tools.

Framework, library, or tool	Modality	Typical tasks / notes
CUDA Toolkit [33]	GPU acceleration	NVIDIA platform comprising compilers, runtime libraries, debuggers, and profilers for GPU computing; commonly used through ML frameworks, with driver and version compatibility as a major operational concern.
cuDNN [34]	GPU acceleration / deep-learning kernels	Optimized NVIDIA primitives for convolutions, normalization, pooling, recurrent operations, and related workloads; a major contributor to training and inference performance on compatible GPUs.
Hugging Face Transformers [35]	Pretrained transformer models	Unified APIs, model implementations, tokenizers, checkpoints, and training utilities for language, vision, audio, and multimodal transformer workflows; facilitates transfer learning and model sharing.
JAX [36]	Differentiable numerical computing	NumPy-oriented programming with automatic differentiation, vectorization, compilation, and accelerator execution; well suited to research requiring composable program transformations and custom high-performance computation.
Keras [37]	Multi-backend deep learning	High-level API for defining and training neural networks across TensorFlow, JAX, and PyTorch backends; supports rapid prototyping, standardized workflows, and selective customization.
LightGBM [38]	Gradient-boosted decision trees	Efficient tree boosting for structured data with histogram-based learning, categorical-feature support, and scalable CPU or GPU execution; strong baseline for tabular prediction.
Optuna [39]	Hyperparameter optimization	Define-by-run search spaces, samplers, pruning, parallel trials, and framework integrations; supports reproducible and resource-efficient tuning from local experiments to distributed studies.
PyTorch [40]	Deep learning	Imperative, Python-centered tensor and automatic-differentiation framework with strong research ergonomics, accelerator support, distributed training, compilation, and a broad domain ecosystem.
scikit-learn [41]	Classical machine learning	Consistent estimator API with mature algorithms for classification, regression, clustering, dimensionality reduction, preprocessing, model selection, and evaluation; particularly strong for tabular baselines.

continued on next page

Framework, library, or tool	Modality	Typical tasks / notes
TensorBoard [42]	Monitoring / visualization	Dashboard environment for scalar metrics, graphs, distributions, embeddings, images, and run comparison; supports diagnosis and communication of training behavior across compatible stacks.
TensorFlow [42]	Deep learning	Eager and compiled execution, automatic differentiation, accelerator support, distributed training, serialization, and an extensive production and deployment ecosystem.
XGBoost [43]	Gradient-boosted decision trees	Scalable, regularized tree boosting with sparse-data support, parallel execution, and mature deployment options; widely used for high-performance structured-data modeling.

5 Computer Vision Frameworks, Libraries, and Tools

Computer-vision tooling combines two historically distinct layers: classical libraries for image formation, filtering, geometry, feature extraction, and calibration, and learning-oriented frameworks for end-to-end detection, segmentation, tracking, and related tasks. In practical systems these layers often coexist, with classical methods handling acquisition, geometric constraints, preprocessing, or postprocessing while learned models provide data-driven representations and predictions.

CV frameworks usually provide more than model definitions. Mature ecosystems include pretrained weights and model zoos, configuration systems, dataset adapters, augmentation pipelines, efficient data loading, established architecture families such as region-based convolutional neural networks (R-CNNs), and evaluation implementations for common benchmarks. Support for Common Objects in Context (COCO)-style annotations and related formats improves interoperability, while export utilities and inference wrappers facilitate deployment. Together, these components reduce duplicated engineering effort and make experiments easier to reproduce and compare.

Because CV workloads are highly sensitive to resolution, augmentation semantics, coordinate conventions, and annotation definitions, interoperability often depends more on data and evaluation consistency than on model architecture alone. Reliable pipelines therefore emphasize explicit dataset schemas, validated transformations, standardized metric computation, and end-to-end profiling from decoding and augmentation through inference and postprocessing.

Table 5: Computer vision frameworks, libraries, and tools.

Framework, library, or tool	Modality	Typical tasks / notes
Albumentations [44]	Image augmentation	Efficient, composable transformations for classification, detection, segmentation, and keypoint tasks; supports synchronized updates of images and annotations and integrates with major deep-learning frameworks.
Detectron2 [45]	Detection and segmentation	Modular PyTorch toolkit with strong implementations of Faster R-CNN, Mask R-CNN, RetinaNet, and related models; configuration-driven experiments, COCO-style datasets, and standardized evaluation.
Kornia [46]	Differentiable computer vision	PyTorch-based differentiable operators for filtering, geometry, camera models, augmentation, and image transformations; enables classical vision operations to participate directly in gradient-based pipelines.
MMDetection [47]	Detection and instance segmentation	OpenMMLab toolbox with modular components, extensive model configurations, pretrained checkpoints, and benchmarking utilities for object detection and instance segmentation.
OpenCV [48]	Classical computer vision	Broad C++ library with Python bindings for filtering, transformations, features, optical flow, calibration, media I/O, and real-time pipelines; commonly used for preprocessing and postprocessing.
Single Shot Multi-Box Detector (SSD) [49]	Object detection	One-stage detector that predicts classes and boxes from multiscale feature maps; historically important for efficient real-time detection and still useful as a compact baseline.
You Only Look Once (YOLO) [50]	Object detection	Influential one-stage detection paradigm optimized for high-throughput inference; subsequent variants provide extensive pretrained models, training utilities, and deployment pathways.

6 Supporting Tools and Ecosystems

End-to-end ML/CV systems depend on substantially more than model-training code. Supporting tools address data collection and labeling, experiment tracking, dataset and model versioning, interpretability, packaging, serving, and monitoring. These components frequently determine whether a project can scale beyond an individual developer and whether its results remain reproducible, auditable, and maintainable.

Supporting ecosystems span data-centric workflows such as annotation, quality control, lineage, and dataset versioning; experiment management for parameters, metrics, artifacts, and checkpoints; optimization through structured hyperparameter search; interpretability and robustness analysis; and operational tooling for packaging, serving, monitoring, and rollback. Because many production failures originate in data drift, pipeline defects, or governance gaps rather than in model architecture, these tools are integral to system reliability.

The following subsections summarize these categories and representative tools within the broader ML/CV lifecycle.

6.1 Annotation and labeling tools

Annotation is often the dominant bottleneck in supervised learning because it is expensive, time-consuming, and difficult to scale without loss of consistency. In CV, labels are not merely ground truth but an operational specification of the task. Ambiguous class definitions, inconsistent boundaries, and undocumented guideline changes introduce noise, limit attainable performance, and compromise benchmark validity. Effective annotation therefore includes taxonomy design, edge-case definitions, annotator training, review procedures, and measurement of inter-annotator agreement or comparable quality indicators.

Open-source tools such as the Computer Vision Annotation Tool (CVAT), LabelImg, and the Visual Geometry Group (VGG) Image Annotator support common annotation types and can be self-hosted, which is advantageous for cost control and privacy-sensitive data. Commercial platforms such as Labelbox and Supervisely add collaboration, audit trails, role-based access, quality assurance (QA) workflows, versioning, and model-assisted labeling. Mature annotation programs treat labeling as a continuous data-engineering process with governed guideline changes, systematic quality sampling, and explicit integration between annotation versions, dataset partitions, training pipelines, and evaluation records.

6.2 Data preprocessing and management

Efficient data management becomes essential as dataset size, heterogeneity, and provenance exceed the capacity of a single workstation. The challenge is not only storing large file collections but also transforming, validating, and serving them without silent corruption, split leakage, or inconsistent preprocessing. Distributed systems such as Apache Spark and the Apache Hadoop ecosystem can parallelize extract, transform, and load (ETL) workloads, join large sources, and compute features or quality summaries across clusters, moving expensive transformations upstream from repeated training runs.

Data themselves must also be versioned and linked to the procedures that produced them. Data Version Control (DVC) applies content-addressed, Git-like workflows to datasets, models, and pipeline artifacts, making changes in cleaning, labeling, or feature extraction traceable. MLflow complements this capability by recording parameters, metrics, artifacts, and model versions. Used together with immutable storage and explicit pipeline definitions, such tools improve provenance and enable experiments to be reconstructed from code, data, configuration, and environment state.

6.3 Hyperparameter optimization

Hyperparameter selection can substantially alter model performance because learning rates, regularization, augmentation, batch size, architectural dimensions, loss weights, and postprocessing thresholds are not learned automatically in standard training. Tuning is also coupled to resource constraints: changes in device count, memory, numerical precision, or distributed execution can modify effective optimization dynamics. Systematic search is therefore important both for obtaining strong baselines and for making fair comparisons between methods.

Optuna, Hyperopt, and Ray Tune support structured search through random, adaptive, and Bayesian strategies, conditional spaces, and pruning or early stopping of unpromising trials. Parallel execution can distribute trials across devices or nodes, while resource-aware schedulers adapt allocation according to intermediate results. These tools are most effective when integrated with configuration management and experiment tracking so that every trial is reproducible and the selected configuration retains a complete provenance record.

6.4 Visualization and model interpretability

Visualization is essential for model diagnosis and scientific communication because many failure modes are more apparent in graphical or qualitative evidence than in aggregate metrics. Training trajectories can reveal instability, overfitting, poor learning-rate schedules, or data-pipeline defects, while inspection of predictions can expose missed small objects, segmentation-boundary artifacts, and spurious reliance on background or acquisition cues. Effective workflows combine scalar metrics with representative predictions, saliency or attention maps, confusion matrices, embedding projections, and data-distribution summaries.

TensorBoard and Weights & Biases support centralized logging, real-time dashboards, and comparison of metrics, hyperparameters, system utilization, and artifacts across runs. Such records are especially valuable when many experiments execute concurrently or when results must later be audited. Standardizing what is logged improves both collaboration and operational discipline and helps distinguish modeling problems from infrastructure or data-pipeline failures.

Interpretability libraries such as SHapley Additive exPlanations (SHAP) and Local Interpretable Model-Agnostic Explanations (LIME) provide mechanisms for examining which inputs, features, or regions contribute to predictions. Explanations do not by themselves establish correctness or causality, but they can reveal leakage, proxy variables, subgroup-specific behavior, or brittle decision rules. Their greatest value arises when they are evaluated systematically on curated failures, relevant subgroups, and distribution shifts rather than used only as illustrative visualizations.

6.5 Deployment and model serving

Dedicated serving frameworks bridge the gap between a trained model and a dependable inference service. TensorFlow Serving and TorchServe provide standardized mechanisms for loading versioned models, exposing inference APIs, batching requests, managing concurrency, and integrating with logging and monitoring. These capabilities reduce custom infrastructure code and support controlled rollouts, rollback, and zero-downtime updates. Reliable serving nevertheless requires preprocessing and postprocessing to be packaged and tested with the model, because mismatches at these boundaries frequently cause production failures.

Model interchange formats and specialized runtimes are increasingly important in heterogeneous environments. The Open Neural Network Exchange (ONNX) defines a common representation for transferring supported models between training frameworks and deployment targets. This can simplify optimized inference on particular devices or standardized serving stacks, but success depends on operator coverage, numerical equivalence, dynamic-shape support, and consistent preprocessing. Export validation should therefore form an explicit stage of the deployment pipeline.

Containerization and orchestration provide the operational foundation for scalable serving. Docker packages a model service with its dependencies so that behavior remains consistent across environments, while Kubernetes manages scheduling, resource allocation, health checks, scaling, rolling updates, and failure recovery. Combined with observability, security controls, and drift monitoring, these technologies support continuous operation under changing load and facilitate controlled lifecycle management.

7 Performance and Benchmarking

Performance evaluation in ML/CV has two complementary objectives: model quality on representative tasks and system efficiency under realistic constraints such as latency, throughput, memory, energy, and cost. Meaningful benchmarking therefore extends beyond a single accuracy measure and considers the complete pipeline from data loading and preprocessing through training or inference to postprocessing and output handling.

Two distinctions are particularly useful. Microbenchmarks isolate operations such as individual kernels and help identify low-level bottlenecks, whereas end-to-end benchmarks capture data preparation, augmentation, synchronization, framework overhead, export, and serving. Standardized evaluation supports comparison across systems, but application-specific assessment must additionally reflect domain metrics, realistic workloads, hardware constraints, and operational stress conditions. Both perspectives are required for responsible interpretation of performance.

Reproducible benchmarking requires documentation of hardware, device memory, drivers, framework and library versions, CUDA and cuDNN versions where applicable, batch size, precision, input dimensions, warm-up procedures, repetition count, and aggregation statistics. Determinism settings and synchronization points should also be reported because they can materially affect timing and variability. In production settings, latency distributions, throughput, peak memory, energy consumption, and monetary cost may be more decisive than small differences in predictive accuracy.

The following subsections summarize commonly reported metrics, benchmark datasets, and principles for comparing frameworks and implementations.

7.1 Performance metrics

Model benchmarking combines task-specific quality measures with explicit choices concerning averaging, thresholds, and evaluation protocols. Identical predictions can yield materially different scores depending on how results are aggregated. Classification studies commonly report accuracy, precision, recall, and the F1 score, supplemented by confusion matrices and calibration analyses when probabilistic outputs are relevant. For imbalanced data, macro-, micro-, and class-weighted averaging, per-class results, and threshold selection should be reported because they can be more informative than a single headline value.

Object-detection evaluation must account for both classification and localization. Mean average precision (mAP) summarizes precision–recall behavior under specified overlap criteria, commonly defined through intersection over union (IoU) between predicted and reference boxes. Reporting multiple IoU thresholds and object-size strata reveals sensitivity to localization precision and small objects. For segmentation, IoU and the Dice coefficient quantify regional agreement, while boundary metrics and per-class scores can expose errors that are obscured by a global average.

Across tasks, scalar measures should be paired with qualitative inspection and slice-based analysis. Aggregate performance can conceal systematic errors affecting particular subgroups, acquisition conditions, or rare classes. Credible benchmarking therefore combines standardized metrics with transparent protocols, uncertainty estimates where appropriate, subgroup breakdowns, and representative examples of both successful and failed predictions.

7.2 Benchmark datasets

Standard benchmark datasets have accelerated progress by providing labeled data, stable task definitions, and widely adopted evaluation protocols. ImageNet, Microsoft Common Objects in Context (MS COCO), and the PASCAL Visual Object Classes (PASCAL VOC) established common reference points and reduced ambiguity concerning partitions, annotation formats, and metrics. Their influence extends beyond leaderboards: they shaped architectures, pretraining strategies, augmentation practices, and transfer-learning workflows in which representations learned at scale are adapted to smaller downstream datasets.

Classical ML relies on analogous repositories and competition platforms. The University of California, Irvine (UCI) Machine Learning Repository provides curated structured datasets used for teaching, algorithm comparison, and baseline evaluation, while Kaggle combines datasets with public competitions and community solutions. Such resources can encourage disciplined feature engineering, cross-validation, and end-to-end pipeline development, although competition-specific practices do not automatically transfer to operational applications.

Benchmarks must nevertheless be interpreted with awareness of their limitations. Popular datasets may become saturated, and repeated optimization can exploit benchmark-specific biases, duplicates, or annotation artifacts. Operational data often differ in prevalence, acquisition, quality, and ambiguity. Strong evaluation therefore combines established benchmarks with domain-specific validation, documented provenance and labeling procedures, leakage checks, and slices that reflect the intended population, environment, and failure modes.

7.3 Framework benchmarking

Framework comparisons often emphasize throughput, memory consumption, and scaling efficiency, but meaningful conclusions require a precisely defined workload. Results depend on architecture, tensor shapes, batch size, precision, kernel maturity, compilation settings, and hardware. Frameworks also differ in scheduling, activation materialization, memory allocation, caching, checkpointing, and communication. Multi-GPU and multi-node measurements introduce additional sensitivity to interconnects, parallelism strategy, synchronization, and input throughput. End-to-end benchmarking in a documented environment is therefore more informative than isolated peak values and should report stability, variability, determinism, and operational complexity alongside speed.

Framework design also shapes developer experience. PyTorch’s imperative execution model closely follows ordinary Python control flow and often simplifies experimentation and debugging. TensorFlow combines eager execution with graph compilation, which can support optimization, portability, and standardized deployment. JAX emphasizes composable program transformations and compilation, while Keras provides a higher-level multi-backend abstraction. The appropriate choice depends on team expertise, existing infrastructure, required control, deployment targets, and the expected lifecycle of the system rather than on a universal ranking.

8 Conclusion and Outlook

Software frameworks, libraries, and tools are essential enablers of innovation in ML and CV because they determine how effectively ideas can be implemented, evaluated, shared, and deployed. Python-centered ecosystems combine productive high-level interfaces with optimized native backends and accelerator support, while frameworks such as PyTorch, TensorFlow, JAX, and Keras support differentiable model development at scale. Classical and specialized CV libraries—including OpenCV, Detectron2, MMDetection, Kornia, and augmentation toolkits—provide reusable components, pretrained models, and standardized evaluation. Supporting systems for annotation, versioning, optimization, interpretability, and serving often determine whether projects remain reproducible and maintainable as they mature.

Future tool development will continue to be shaped by transformer-based and multimodal models, efficient training and inference, compilation, and increasing automation of experimentation and software modification. Privacy, security, and governance requirements will place greater emphasis on provenance, auditability, controlled data access, and safe distributed workflows, while benchmarking will increasingly consider robustness, energy use, latency, and cost alongside accuracy. Practitioners should therefore assess tools not only by immediate convenience but also by interoperability, abstraction quality, ecosystem stability, governance controls, and deployment support, reinforced by disciplined environment management, version control, testing, documentation, and monitoring.

References

- [1] N. Maslej, L. Fattorini, C. R. Perrault, V. Parli, A. Reuel, E. Brynjolfsson, J. Etchemendy, K. Ligett, T. Lyons, J. Manyika, J. C. Niebles, Y. Shoham, R. Wald, and J. Clark, “The AI index 2024 annual report,” Stanford Institute for Human-Centered Artificial Intelligence, Stanford, CA, USA, Tech. Rep., 2024. [Online]. Available: <https://aiindex.stanford.edu/report/>
(2 citations on 2 pages: 6 and 7)
- [2] D. Kreuzberger, N. Köhl, and S. Hirschl, “Machine learning operations (MLOps): Overview, definition, and architecture,” *IEEE Access*, vol. 11, pp. 31 866–31 879, 2023. [Online]. Available: <https://doi.org/10.1109/ACCESS.2023.3262138>
(1 citation on 1 page: 8)
- [3] E. Bisong, *Google Colaboratory*. Berkeley, CA, USA: Apress, 2019, pp. 59–64. [Online]. Available: https://doi.org/10.1007/978-1-4842-4470-8_7
(1 citation on 1 page: 10)
- [4] T. Kluyver, B. Ragan-Kelley, F. Pérez, B. Granger, M. Bussonnier, J. Frederic, K. Kelley, J. Hamrick, J. Grout, S. Corlay, P. Ivanov, D. Avila, S. Abdalla, and C. Willing, “Jupyter notebooks—a publishing format for reproducible computational workflows,” in *Positioning and Power in Academic Publishing: Players, Agents and Agendas*. IOS Press, 2016, pp. 87–90. [Online]. Available: <https://doi.org/10.3233/978-1-61499-649-1-87>
(1 citation on 1 page: 10)
- [5] J. M. Perkel, “Why jupyter is data scientists’ computational notebook of choice,” *Nature*, vol. 563, no. 7729, pp. 145–146, 2018. [Online]. Available: <https://doi.org/10.1038/d41586-018-07196-1>
(1 citation on 1 page: 10)
- [6] G. Preda, *Developing Kaggle Notebooks: Pave Your Way to Becoming a Kaggle Notebooks Grandmaster*. Packt Publishing, 2023. [Online]. Available: <https://subscription.packtpub.com/book/data/9781805128519>
(1 citation on 1 page: 10)
- [7] The MathWorks, Inc., “MATLAB,” Numerical-computing environment, 2026, accessed: 2026-08-20. [Online]. Available: <https://www.mathworks.com/products/matlab.html>
(1 citation on 1 page: 10)
- [8] Q. N. Islam, *Mastering PyCharm*. Packt Publishing, 2015. [Online]. Available: <https://www.packtpub.com/en-us/product/mastering-pycharm-9781783551316>
(1 citation on 1 page: 10)
- [9] RStudio Team, *RStudio: Integrated Development Environment for R*, RStudio, PBC, Boston, MA, USA, 2020. [Online]. Available: <https://posit.co/products/open-source/rstudio/>
(1 citation on 1 page: 11)
- [10] P. Raybaut, “Spyder: Scientific python development environment,” Software and documentation, 2009. [Online]. Available: <https://www.spyder-ide.org/>
(1 citation on 1 page: 11)

- [11] S. Amann, S. Proksch, S. Nadi, and M. Mezini, “A study of visual studio usage in practice,” in *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, vol. 1, 2016, pp. 124–134. [Online]. Available: <https://doi.org/10.1109/SANER.2016.39> (1 citation on 1 page: 11)
- [12] J. Tan, Y. Chen, and S. Jiao, “Visual studio code in introductory computer science course: An experience report,” *arXiv preprint arXiv:2303.10174*, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2303.10174> (1 citation on 1 page: 11)
- [13] Amazon Web Services, “Using Amazon Q Developer in the IDE,” 2026, accessed: 2026-08-20. [Online]. Available: <https://docs.aws.amazon.com/amazonq/latest/qdeveloper-ug/q-in-IDE.html> (1 citation on 1 page: 13)
- [14] R. Khojah, M. Mohamad, P. Leitner, and F. G. de Oliveira Neto, “Beyond code generation: An observational study of ChatGPT usage in software engineering practice,” *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 1819–1840, 2024. [Online]. Available: <https://doi.org/10.1145/3660788> (1 citation on 1 page: 13)
- [15] Anthropic, “Claude Code: Overview,” 2026, accessed: 2026-08-20. [Online]. Available: <https://code.claude.com/docs/en/overview> (1 citation on 1 page: 13)
- [16] S. Jiang and D. Nam, “Beyond the prompt: An empirical study of Cursor rules,” *arXiv preprint arXiv:2512.18925*, 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2512.18925> (1 citation on 1 page: 13)
- [17] Google, “Use the Gemini Code Assist chat,” 2026, accessed: 2026-08-20. [Online]. Available: <https://docs.cloud.google.com/gemini/docs/codeassist/use-gemini-code-assist-chat> (1 citation on 1 page: 14)
- [18] B. Zhang, P. Liang, X. Zhou, A. Ahmad, and M. Waseem, “Practices and challenges of using GitHub Copilot: An empirical study,” *arXiv preprint arXiv:2303.08733*, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2303.08733> (1 citation on 1 page: 14)
- [19] A. Sokolov, “Ai assistant in jetbrains ide: Insights and challenges,” in *Proceedings of the 1st ACM International Conference on AI-Powered Software*, 2024, pp. 178–178. [Online]. Available: <https://doi.org/10.1145/3664646.3676273> (1 citation on 1 page: 14)
- [20] V. Corso, L. Mariani, D. Micucci, and O. Riganelli, “Generating java methods: An empirical assessment of four ai-based code assistants,” in *Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension*, 2024, pp. 13–23. [Online]. Available: <https://doi.org/10.1145/3643916.3644402> (1 citation on 1 page: 14)
- [21] M. Rocklin, “Dask: Parallel computation with blocked algorithms and task scheduling,” in *Proceedings of the 14th Python in Science Conference*, 2015, pp. 130–136. [Online]. Available: <https://doi.org/10.25080/Majora-7b98e3ed-013> (1 citation on 1 page: 16)

- [22] J. D. Hunter, “Matplotlib: A 2D graphics environment,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007. [Online]. Available: <https://doi.org/10.1109/MCSE.2007.55>
(1 citation on 1 page: 16)
- [23] A. A. Hagberg, D. A. Schult, and P. J. Swart, “Exploring network structure, dynamics, and function using NetworkX,” in *Proceedings of the 7th Python in Science Conference*, 2008, pp. 11–15. [Online]. Available: <https://doi.org/10.25080/tcwv9851>
(1 citation on 1 page: 16)
- [24] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, no. 7825, pp. 357–362, 2020. [Online]. Available: <https://doi.org/10.1038/s41586-020-2649-2>
(1 citation on 1 page: 16)
- [25] J. Neider, T. Davis, and M. Woo, *OpenGL Programming Guide*. Reading, MA, USA: Addison-Wesley, 1993. [Online]. Available: <https://www.cs.kent.edu/~farrell/graphics/opengl/openglguide/56>
(1 citation on 1 page: 16)
- [26] W. McKinney, “Data structures for statistical computing in python,” in *Proceedings of the 9th Python in Science Conference*, 2010, pp. 51–56. [Online]. Available: <https://doi.org/10.25080/Majora-92bf1922-00a>
(1 citation on 1 page: 16)
- [27] N. Kruchten, A. Seier, and C. Parmer, “Plotly.py: An interactive, open-source, and browser-based graphing library for python,” 2024, computer software, version 5.24.1. [Online]. Available: <https://doi.org/10.5281/zenodo.14503524>
(1 citation on 1 page: 16)
- [28] S. van der Walt, J. L. Schönberger, J. Nunez-Iglesias, F. Boulogne, J. D. Warner, N. Yager, E. Gouillart, T. Yu, and scikit-image contributors, “scikit-image: Image processing in python,” *PeerJ*, vol. 2, p. e453, 2014. [Online]. Available: <https://doi.org/10.7717/peerj.453>
(1 citation on 1 page: 16)
- [29] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright *et al.*, “SciPy 1.0: Fundamental algorithms for scientific computing in python,” *Nature Methods*, vol. 17, pp. 261–272, 2020. [Online]. Available: <https://doi.org/10.1038/s41592-019-0686-2>
(1 citation on 1 page: 16)
- [30] M. L. Waskom, “seaborn: Statistical data visualization,” *Journal of Open Source Software*, vol. 6, no. 60, p. 3021, 2021. [Online]. Available: <https://doi.org/10.21105/joss.03021>
(1 citation on 1 page: 16)
- [31] S. Seabold and J. Perktold, “Statsmodels: Econometric and statistical modeling with python,” in *Proceedings of the 9th Python in Science Conference*, 2010, pp. 92–96. [Online]. Available: <https://doi.org/10.25080/Majora-92bf1922-011>
(1 citation on 1 page: 17)

- [32] A. Meurer, C. P. Smith, M. Paprocki, O. Čertík, S. B. Kirpichev, M. Rocklin, A. Kumar, S. Ivanov, J. K. Moore, S. Singh *et al.*, “SymPy: Symbolic computing in python,” *PeerJ Computer Science*, vol. 3, p. e103, 2017. [Online]. Available: <https://doi.org/10.7717/peerj-cs.103>
(1 citation on 1 page: 17)
- [33] J. Nickolls, I. Buck, M. Garland, and K. Skadron, “Scalable parallel programming with CUDA,” *ACM Queue*, vol. 6, no. 2, pp. 40–53, 2008. [Online]. Available: <https://doi.org/10.1145/1365490.1365500>
(1 citation on 1 page: 19)
- [34] S. Chetlur, C. Woolley, P. Vandermersch, J. Cohen, J. Tran, B. Catanzaro, and E. Shelhamer, “cuDNN: Efficient primitives for deep learning,” *arXiv preprint arXiv:1410.0759*, 2014. [Online]. Available: <https://doi.org/10.48550/arXiv.1410.0759>
(1 citation on 1 page: 19)
- [35] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. Le Scao, S. Gugger, M. Drame, Q. Lhoest, and A. M. Rush, “Transformers: State-of-the-art natural language processing,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, 2020, pp. 38–45. [Online]. Available: <https://doi.org/10.18653/v1/2020.emnlp-demos.6>
(1 citation on 1 page: 19)
- [36] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, and S. Wanderman-Milne, “JAX: Composable transformations of Python+NumPy programs,” Software, 2018. [Online]. Available: <https://github.com/jax-ml/jax>
(1 citation on 1 page: 19)
- [37] F. Chollet *et al.*, “Keras,” Software, 2015. [Online]. Available: <https://keras.io/>
(1 citation on 1 page: 19)
- [38] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “LightGBM: A highly efficient gradient boosting decision tree,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 3146–3154. [Online]. Available: <https://proceedings.neurips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html>
(1 citation on 1 page: 19)
- [39] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. Association for Computing Machinery, 2019, pp. 2623–2631. [Online]. Available: <https://doi.org/10.1145/3292500.3330701>
(1 citation on 1 page: 19)
- [40] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019, pp. 8024–8035. [Online]. Available: <https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html>
(1 citation on 1 page: 19)

- [41] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. VanderPlas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, “Scikit-learn: Machine learning in python,” *Journal of Machine Learning Research*, vol. 12, no. 85, pp. 2825–2830, 2011. [Online]. Available: <https://www.jmlr.org/papers/v12/pedregosa11a.html>
(1 citation on 1 page: 19)
- [42] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin *et al.*, “TensorFlow: Large-scale machine learning on heterogeneous systems,” Software available from tensorflow.org, 2015. [Online]. Available: <https://www.tensorflow.org/>
(2 citations on 1 page: 20)
- [43] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. Association for Computing Machinery, 2016, pp. 785–794. [Online]. Available: <https://doi.org/10.1145/2939672.2939785>
(1 citation on 1 page: 20)
- [44] A. Buslaev, V. I. Iglovikov, E. Khvedchenya, A. Parinov, M. Druzhinin, and A. A. Kalinin, “Albumentations: Fast and flexible image augmentations,” *Information*, vol. 11, no. 2, p. 125, 2020. [Online]. Available: <https://doi.org/10.3390/info11020125>
(1 citation on 1 page: 22)
- [45] Y. Wu, A. Kirillov, F. Massa, W.-Y. Lo, and R. Girshick, “Detectron2,” Software, 2019. [Online]. Available: <https://github.com/facebookresearch/detectron2>
(1 citation on 1 page: 22)
- [46] E. Riba, D. Mishkin, D. Ponsa, E. Rublee, and G. Bradski, “Kornia: An open source differentiable computer vision library for PyTorch,” in *2020 IEEE Winter Conference on Applications of Computer Vision*, 2020, pp. 3663–3672. [Online]. Available: <https://doi.org/10.1109/WACV45572.2020.9093363>
(1 citation on 1 page: 22)
- [47] K. Chen, J. Wang, J. Pang, Y. Cao, Y. Xiong, X. Li, S. Sun, W. Feng, Z. Liu *et al.*, “MMDetection: Open MMLab detection toolbox and benchmark,” *arXiv preprint arXiv:1906.07155*, 2019. [Online]. Available: <https://doi.org/10.48550/arXiv.1906.07155>
(1 citation on 1 page: 22)
- [48] G. Bradski, “The OpenCV library,” *Dr. Dobbs’s Journal of Software Tools*, vol. 25, no. 11, pp. 120, 122–125, Nov. 2000. [Online]. Available: <https://jacobfilipp.com/DrDobbs/articles/DDJ/2000/0011/0011k/0011k.htm>
(1 citation on 1 page: 22)
- [49] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, “SSD: Single shot multibox detector,” in *Proceedings of the European Conference on Computer Vision*. Springer, 2016, pp. 21–37. [Online]. Available: https://doi.org/10.1007/978-3-319-46448-0_2
(1 citation on 1 page: 22)
- [50] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 779–788. [Online]. Available: <https://doi.org/10.1109/CVPR.2016.91>
(1 citation on 1 page: 22)